Agrovesto Infrastructure: Technical Documentation

Single source of truth for the `Agrovesto/infrastructure` repository. Version 1.0 · 24 September 2026 · Region `eu-west-1` (Ireland)

> Where this document and the code disagree, the code wins. Update this file in the same PR as the change. Step-by-step import commands are in `docs/import-runbook.md`. Cost details are in `docs/infracost.md`. Diagrams are in `docs/architecture/`.

1. Overview

Agrovesto runs on AWS in two accounts, both managed with Terraform from this repository.

Account	ID	Terraform stacks	Purpose
Dev	`353833416626`	`bootstrap` (dev.tfvars), `dev`	Development: single EC2 host running every API, SQL Server and logging in Docker; Amplify frontends
Prod	`221434544570`	`bootstrap` (prod.tfvars), `prod`, `monitoring`	Production: ECS Fargate APIs behind an ALB, RDS SQL Server, Vault; plus the observability stack

Design principles

- **Hybrid adoption.** Existing AWS resources that the new design uses are imported (`imports.tf`), everything else is created fresh, and legacy resources are left unmanaged to be deleted by hand later. There are no duplicates.
- **No secrets in code.** Secrets come from Secrets Manager or GitHub secrets. Existing credentials are read through data sources.
- **Everything is tested.** Every stack has a `terraform test` suite with mock providers, gated in CI at 95% resource coverage (currently 100%).
- **Everything is deployed through Git.** PR = checks + plan. Merge = apply.

2. Repository layout

```
infrastructure/
├── bootstrap/          State bucket + GitHub OIDC provider + deploy role (per account)
│   ├── dev.tfvars      Values for the dev account
│   └── prod.tfvars     Values for the prod account
├── dev/                Dev account stack (EC2 host, Amplify, ECR, SNS, Rekognition)
├── prod/               Prod account application stack (VPC, ECS, ALB, RDS, Vault, SNS, Rekognition)
├── monitoring/         Prod account observability stack (Prometheus, Loki, Tempo, Grafana, ...)
│   └── tests/          (every stack has tests/*.tftest.hcl)
├── workflows/          CI/CD source: copy into .github/workflows/
├── scripts/            tf_coverage.py, infracost.sh, run_test_suite.sh
├── infracost.yml, infracost-usage/, infracost-extras.json
├── sonar-project-{dev,prod,monitoring}.properties
└── docs/               This document, import runbook, infracost notes, architecture diagrams
```

Each stack directory is a flat, self-contained Terraform root module: `versions.tf` (backend + providers), `variables.tf`, `terraform.auto.tfvars` (real values), `locals.tf`, `outputs.tf`, one file per concern, `imports.tf` (one-shot adoption), `checkov.yml`, `trivy.yml`, `tests/`.

Tooling versions: Terraform `>= 1.7` (CI pins `1.10.5`), AWS provider `~> 5.40` in dev and prod (`~> 5.0` elsewhere), random `~> 3.6`.

---

3. Branches, environments and the promotion flow

Branch	Role	Workflow	PR opens / updates	Merge / push
`devops`	Playground	`devops.yml`	All checks for dev, prod and monitoring (no AWS access)	Same, no deploy
`dev`	Dev account	`dev.yml`	Full checks + plan for `dev`	**Deploys dev**
`main`	Prod account	`prod.yml`	Full checks + plan for `prod`, then `monitoring`	**Deploys prod, then monitoring** (after `production` approval)

Recommended flow: feature branch → PR into `devops` (experiment) → PR into `dev` (deploys to dev) → PR from `dev` into `main` (deploys to prod).

Stack apply order (first rollout and any rebuild): `bootstrap → dev → prod → monitoring`. Monitoring reads prod's VPC, ALB and hosted zone through data sources, so it always goes after prod.

---

4. Terraform state

Stack	Bucket	Key	Workspace
bootstrap	local state (per account, run by an admin)	n/a	n/a
dev	`nomaos-tfstate-353833416626-eu-west-1`	`dev/bootstrap.tfstate`	`default`
prod	`nomaos-tfstate-221434544570-eu-west-1`	`prod/bootstrap.tfstate`	`default`
monitoring	`nomaos-tfstate-221434544570-eu-west-1`	`prod/bootstrap.tfstate`	**`monitoring`**

- Prod and monitoring share one key and are separated by workspace. CI sets `TF_WORKSPACE=monitoring` for the monitoring stack. Locally, run `terraform workspace select monitoring` before running monitoring commands.
- The state buckets are versioned, encrypted, block public access and have lifecycle rules. The dev bucket uses SSE-KMS with the key `31f57b1f-…`, and non-current versions are kept for 365 days.

---

5. Bootstrap stack

Run once per account from a workstation with admin credentials:

```
cd bootstrap
terraform init
terraform plan  -var-file=dev.tfvars    # or prod.tfvars
terraform apply -var-file=dev.tfvars
```

Resource	Dev	Prod
State bucket	imported	imported
GitHub OIDC provider `token.actions.githubusercontent.com`	imported	imported
Deploy role	`nomaos-dev-github-terraform` (imported)	`agrovesto-prod-github-deploy` (imported)

Resource	Dev	Prod
Deploy permissions	`AdministratorAccess` attachment, imported (`deploy_admin_access = true`)	Scoped policy `<role>-policy` (`deploy_admin_access = false`)

OIDC trust (`github_sub_patterns`). The deploy roles accept tokens from `main`, `dev`, `arena/*`, pull requests and the GitHub Environments `dev` / `production`. Jobs that use an Environment present `…:environment:<name>` as their subject, which is why those entries are required.

---

6. Dev stack (account 353833416626)

6.1 Network

Item	Value	Managed how
VPC	`vpc-0d9b566c353787d53`, `172.96.0.0/16`	imported
Internet gateway	`igw-04e4d65d903b2b67d`	imported
Public subnet	`subnet-0739cfd4c412deb3f`, `172.96.0.0/24`, eu-west-1a	imported
Route table	`rtb-018e7f1b9df073189` (0.0.0.0/0 → IGW)	imported
Default SG	`sg-032197de8d143373f`: **all rules removed** when managed	imported (`manage_default_security_group`)
NAT	none (by design)	n/a

6.2 Application host

One EC2 instance (`t3.large`, IMDSv2 only, hop limit 2) runs everything in Docker Compose:

- **APIs** (images from ECR `agrovesto-dev-<svc>`): wallet :5001, onboarding :5002, input :5003, moneval :5004, serviceprovider :5005
- **SQL Server** on its own EBS volume, with databases `wallet`, `onboarding`, `input`, `moneval`, `serviceprovider`. The SA password comes from the `TF_VAR_MSSQL_SA_PASSWORD_DEV` secret.
- **nginx** reverse proxy: `dev-<svc>-api.nomaos.com` → API ports. Security group: 80/443 open, 22 from `allowed_ssh_cidrs` only.
- **Logging and metrics** (own EBS volume): Loki 3.4.2, Alloy 1.6.1, Grafana 11.5.2, Prometheus 3.1.0, cAdvisor 0.57.0, node-exporter 1.9.1, **Alertmanager 0.28.1**. All bind to 127.0.0.1. Grafana is exposed only through the nginx allow-list.

How configuration reaches the host. Rendered files (compose, nginx, SQL init, Loki, Alloy, Prometheus, alert rules, Alertmanager and Grafana provisioning) are uploaded as objects to the host-config S3 bucket. `user_data` syncs them at boot. After changing config, run `aws s3 sync` on the host, or reboot it.

Scheduler. EventBridge Scheduler starts the host at 08:00 and stops it at 20:00, Monday to Friday, in the Africa/Lagos timezone.

Backups. A private, versioned S3 backup bucket holds SQL Server backups.

Legacy host. `i-00af32231113d64f9` is not managed. At cut-over, move Elastic IP `eipalloc-0b7f04c0906f47a4e` to the new host, then retire the old one.

6.3 Frontends (Amplify)

Key	App ID	Repo	Branch	Subdomain
onboarding	`d3v79u00ot9glq` (imported)	Agrovesto/onboarding-frontend	dev	onboarding-dev
landing	`dfvk5chhp2iln` (imported)	Agrovesto/nomaos-landingpage	staging	dev-agrovesto

`build_spec`, `environment_variables` and tokens are in `ignore_changes`, so adoption never overwrites the live build settings.

6.4 DNS

`manage_dns = false`: the `nomaos.com` zone lives in prod, and dev records are created there.

---

7. Prod stack (account 221434544570)

7.1 Network

Tier	Subnets	Routing	Status
VPC	`vpc-0016f521bae08f4ff`, `172.96.0.0/16`; IGW `igw-07275a1a62726a4c8`	n/a	imported
Public	`subnet-0b808deeaf8391867` (172.96.0.0/24, 1a), `subnet-0e169f40c70cdfe8b` (172.96.1.0/24, 1b)	shared RT `rtb-0530eb261b38f735b` → IGW	imported (the second subnet is re-associated to the shared table)
Private (ECS)	172.96.11.0/24, 172.96.12.0/24	single NAT gateway	**new**
Database	`subnet-00ee6901e53783c86` (RT `rtb-0fa42099da11b78e9`), `subnet-084f21457f316c44c` (RT `rtb-084528a52ec739f38`)	private	imported

VPC flow logs are enabled. The default SG is **not** managed (`manage_default_security_group = false`) because it is in use.

7.2 Compute: ECS Fargate

- One cluster. Five services: wallet :5001, onboarding :5002, input :5003, moneval :5004, serviceprovider :5005.
- 256 CPU / 512 MiB each, ARM64 (x86 fallback available), desired count 1, autoscaling 1–5 at 1000 requests per task.
- Tasks run in the private subnets (`single_az_ecs = true`). Images come from ECR `agrovesto-prod-<svc>`, which is IMMUTABLE and keeps 5 tagged images.
- **HashiCorp Vault** runs as its own ECS service with storage on EFS and KMS auto-unseal, at `vault.nomaos.com`.
- **ALB** `agrovesto-prod-alb`: HTTPS 443 with the ACM wildcard `*.nomaos.com`; port 80 redirects to 443. Host rules send `<svc>-api.nomaos.com` to each service and `vault.nomaos.com` to Vault. Access logs go to S3.

7.3 Database: RDS SQL Server

Setting	Value
Instance	`migration-prod` (imported), `sqlserver-ex 15.00.4445.1.v1`, `db.t3.micro`
Storage	50 GB gp2, autoscaling to 63 GB, **unencrypted** (as live; encrypting it would force a replacement)
Subnet group	`agxdb2subnetgroup` (imported)
Security groups	design SG + existing `sg-05a2722a050c1109b`, `sg-0b555edfc6d445a18`
Master credentials	Existing secret **`nomaosDBcredentials`**, read by the apps. RDS-managed rotation is off (`mssql_manage_master_user_password = false`)
Monitoring	Enhanced monitoring via `rds-monitoring-role`; Performance Insights on
Windows	backup 04:13–04:43 UTC, maintenance Mon 22:43–23:13 UTC
Multi-AZ	off

A one-shot `null_resource.db_init` creates the application databases.

7.4 DNS and certificates

Hosted zones: `nomaos.com` `Z0208643NZ0W8WF5RQ9F` and `agrovesto` `Z00055662LTUSSMS5ZA84` (both imported). ACM certificates are DNS-validated in these zones.

---

8. Monitoring stack (prod account)

Runs on ECS Fargate in the prod VPC (discovered through data sources). Storage is on EFS, encrypted with a dedicated KMS key. Service discovery uses Cloud Map (`monitoring.internal`).

Component	Purpose	Retention
Prometheus + Alertmanager	Metrics, alert rules, routing to SNS `agrovesto-prod-alerts`	15 days
Loki	Logs	720 h (30 days)
Tempo	Traces (span metrics to Prometheus)	336 h (14 days)
Grafana	Dashboards at `grafana.nomaos.com` through the prod ALB (VPC-only unless `grafana_allowed_cidrs` is set)	n/a
Alloy	Collection agent	n/a
Blackbox exporter	HTTPS uptime probes: the 5 APIs, Vault `/v1/sys/health`, Grafana `/api/health`	n/a
Pushgateway	Metrics from batch jobs	n/a
RDS collector	Scheduled SQL Server size metrics for `migration-prod`	n/a
Cost	CUR bucket, AWS Budget $250/month (alert at 80%)	n/a

Component	Purpose	Retention
Monthly report	PDF emailed through SES to `tech@agrovesto.com`	monthly

9. Alerting and notifications

```
CloudWatch alarms ─┐
Dev Alertmanager ──┼──► SNS <stack>-alerts (KMS CMK) ──► email tech@agrovesto.com
Prod Alertmanager ─┘        (dev topic / prod topic)
Apps ──────────────────► SNS <stack>-notifications (app topic)
```

Source	Alerts
Dev CloudWatch	host status check failed, host CPU high
Dev Prometheus rules	TargetDown (5 min), HostHighCpu > 90%, HostHighMemory > 90%, HostDiskAlmostFull < 10% free, ContainerDown, ContainerRestarting (> 2 restarts in 15 min)
Prod CloudWatch	RDS CPU / free storage / freeable memory, ALB 5xx, per-service target 5xx, unhealthy targets, ECS CPU, ECS memory
Monitoring Alertmanager	Everything Prometheus evaluates in prod (routes to the prod alerts topic)

- Topics are encrypted with a customer-managed KMS key, because CloudWatch cannot publish to topics that use `aws/sns`.
- **Every email subscription must be confirmed** with the link AWS sends. Until then, alerts are not delivered.
- Dev Alertmanager signs requests with the EC2 instance role (IMDSv2, hop limit 2). No keys are stored.

10. Amazon Rekognition (dev and prod)

Defined in `rekognition.tf`, identical in both stacks. The app role is the EC2 role in dev and the ECS task role in prod.

Feature	Resources
Image analysis (labels, text, moderation, faces, PPE, celebrities, custom labels)	IAM only (`rekognition:Detect*`, `CompareFaces`, ...)
Face collections / KYC	`aws_rekognition_collection.this`, default `<stack>-customers`. Index, search and user actions are scoped to the collection ARNs
Face Liveness	The backend creates the session and reads the result. The client SDK streams with a **Cognito guest identity pool** whose role can only call `StartFaceLivenessSession`
Video analysis	SNS `<stack>-rekognition-video` → SQS queue (+ DLQ, 5 receives). Rekognition publishes through the role `<stack>-rekognition-video`; the app may `iam:PassRole` it to Rekognition only
Media bucket	`<stack>-rekognition-<account>`: private, SSE-S3, HTTPS-only. `uploads/` expires after 90 days, `liveness/` after 30

App integration values: `terraform output rekognition` (collection IDs, bucket, video topic ARN, video role ARN, queue URL, identity pool ID). Start* video calls pass `NotificationChannel = { SNSTopicArn, RoleArn }` from that output. Settings: `var.rekognition` (`collections`, `liveness_identity_pool`, `liveness_retention_days`,

`upload_retention_days`).

11. Security model

Area	Control
CI access to AWS	GitHub OIDC → deploy role. No long-lived keys
Secrets	Secrets Manager (`nomaosDBcredentials`), GitHub secrets (`TF_VAR_MSSQL_SA_PASSWORD_DEV`), compose `.env` on the host. Never in `.tf` files
Encryption	KMS CMKs for SNS, Vault and monitoring (EFS/S3); SSE on all buckets. RDS `migration-prod` is unencrypted (known gap, see §16)
Buckets	Public access blocked and TLS-only policies on every bucket
Instances	IMDSv2 required; SSH restricted to `allowed_ssh_cidrs`; monitoring UIs on loopback or VPC only
Static analysis	Checkov and Trivy (CRITICAL/HIGH fail the build), SonarQube quality gate
Prod change control	`production` GitHub Environment with required reviewers

12. CI/CD

Source files live in `workflows/` and are copied into `.github/workflows/` (the automation account cannot push to `.github/workflows`).

File	Purpose
`terraform-pipeline.yml`	Reusable pipeline for one stack
`dev.yml`	Triggers on any PR into / push to `dev`
`prod.yml`	Triggers on any PR into / push to `main` (prod, then monitoring with `workspace: monitoring`)
`devops.yml`	Triggers on `devops`: checks for all three stacks, no AWS access

Pipeline jobs

1. **Git metadata**: event, who raised the PR or pushed, source → target branch, merge commit, and a table of every commit (SHA, author, date, message) in the job summary.
2. **Checkov + Trivy**: uses the stack's `checkov.yml` / `trivy.yml`. Reports are uploaded as artifacts.
3. **Test and coverage**: `terraform test` plus `scripts/tf_coverage.py --fail-under 95`.
4. **SonarQube**: quality gate (skipped with a notice if `SONAR_TOKEN` is missing).
5. **Infracost**: cost breakdown; on PRs, a cost diff comment against the target branch.
6. **fmt / validate / plan**: `terraform fmt -check`, `validate`, `plan -detailed-exitcode`, then a diff table (import / create / update / delete / REPLACE) and the full plan in one updating PR comment.
7. **Apply** (push only, and only when the plan has changes): applies the saved `tfplan` artifact inside the GitHub Environment.

Required repository secrets: `AWS_ROLE_ARN_DEV`, `AWS_ROLE_ARN_PROD`, `TF_VAR_MSSQL_SA_PASSWORD_DEV`, `SONAR_TOKEN`, `SONAR_HOST_URL` (optional), `INFRACOST_API_KEY`. **Required Environments:** `dev`, `production` (with reviewers).

13. Testing

- `terraform test` with `mock_provider "aws"`: no credentials, no AWS calls.
- Import blocks are disabled in tests (`import_existing = false`), because mock providers cannot import.
- Use `command = plan` by default. Assert only on values known at plan time; mocked ARNs are random, so JSON policies containing ARNs are unknown in plan. Use `command = apply` only when the stack tolerates random ARNs (prod does; dev does not, because of ARN validation on alarms, the scheduler and SNS).
- Coverage = resource/data blocks referenced by at least one test ÷ all blocks.

Stack	Tests	Coverage
dev	34	100% (81/81)
prod	64	100% (133/133)
monitoring	24	100% (94/94)

Run locally:

```
for s in dev prod monitoring; do
 (cd $s && terraform init -upgrade -backend=false >/dev/null && terraform test -no-color)
done
python3 scripts/tf_coverage.py dev --fail-under 95
```

---

14. First rollout procedure (summary)

Full commands are in `docs/import-runbook.md`.

1. **bootstrap** in each account: the plan must show only imports and in-place updates.
2. **dev**: PR into `dev` → review the plan comment → merge. Before merging, check that nothing uses the default SG `sg-032197de8d143373f` (its rules will be removed): `aws ec2 describe-network-interfaces --filters Name=group-id,Values=sg-032197de8d143373f`
3. **prod**: PR into `main` → review → merge → approve the `production` environment.
4. **monitoring**: runs automatically after prod.

Gate for every apply: 0 to destroy, nothing marked REPLACE. Imported rows may only show tag or intended changes. Stop and investigate otherwise.

After each stack plans with No changes, delete its `imports.tf` and the `import_existing` variable.

---

15. Operations

Task	How
Deploy a change	PR into `dev` → merge; then PR `dev` → `main` → merge + approve
Add an API	Add it to `apis` (dev) / `services` (prod) in `terraform.auto.tfvars`; ECR repo, compose/ECS service, DNS and alarms follow
Add an alert recipient	Add it to `alert_emails` (dev/prod `terraform.auto.tfvars`), merge, and have the recipient confirm the email
Change dev alert thresholds	`alert_rules = { cpu_percent = …, memory_percent = …, disk_free_percent = … }`, then resync config on the host
Add a face collection	`rekognition = { collections = ["customers", "staff"] }`
Rotate the prod DB password	Update `nomaosDBcredentials` in Secrets Manager and on RDS; restart the ECS services
Refresh dev host config	SSM into the host → `aws s3 sync s3://<host-config-bucket> /opt/agrovesto` → `docker compose up -d`
Cost report	`./scripts/infracost.sh`
Break-glass local plan	`aws sso login` / profile → `terraform init` → (`terraform workspace select monitoring`) → `terraform plan`

Legacy cleanup (after cut-over). Resources that were intentionally not imported (the old dev host, the extra prod route table `rtb-0614…`, the DMS endpoint referencing the deleted `agxdb2` secret, old Amplify apps such as `tradeplatform`, and others) are listed in section 5 of the runbook. Delete them by hand once the new stacks are live.

---

16. Known risks and open items

Item	Impact	Action
RDS `migration-prod` unencrypted	Compliance gap	Plan a snapshot → encrypted copy → restore migration
Single NAT, single-AZ ECS, no RDS Multi-AZ	Reduced availability (cost trade-off)	Revisit when traffic grows
Dev default SG rules removed on first apply	Anything using it loses access	Check ENIs first (§14)
SNS email confirmations	No alerts until confirmed	Confirm after the first apply
SES sandbox	Monthly report only reaches verified addresses	Request production access or verify recipients
Prod task images are placeholders (`ealen/echo-server`)	Not the real APIs	Set real image tags per service
`nomaosDBcredentials` key names	Apps must read the right keys	Confirm with `jq 'keys'`

---

17. Reference: resource IDs

Account	Resource	ID
dev	VPC / IGW	`vpc-0d9b566c353787d53` / `igw-04e4d65d903b2b67d`
dev	Subnet / RT / default SG	`subnet-0739cfd4c412deb3f` / `rtb-018e7f1b9df073189` / `sg-032197de8d143373f`
dev	Amplify	`d3v79u00ot9glq`, `dfvk5chhp2iln`
dev	Legacy host / EIP	`i-00af32231113d64f9` / `eipalloc-0b7f04c0906f47a4e`
dev	Deploy role	`nomaos-dev-github-terraform`
prod	VPC / IGW	`vpc-0016f521bae08f4ff` / `igw-07275a1a62726a4c8`
prod	Public subnets	`subnet-0b808deeaf8391867`, `subnet-0e169f40c70cdfe8b` (RT `rtb-0530eb261b38f735b`)
prod	DB subnets	`subnet-00ee6901e53783c86`, `subnet-084f21457f316c44c`
prod	Zones	`Z0208643NZ0W8WF5RQ9F` (nomaos.com), `Z00055662LTUSSMS5ZA84` (agrovesto)
prod	RDS	`migration-prod`, subnet group `agxdb2subnetgroup`
prod	Extra SGs	`sg-05a2722a050c1109b`, `sg-0b555edfc6d445a18`
prod	DB secret	`nomaosDBcredentials`
prod	Deploy role	`agrovesto-prod-github-deploy`

Contacts: alerts and reports go to `tech@agrovesto.com`.